



SafeCross

מערכת חכמה מבוססת AI לבטיחות מעברי חצייה

מסמך אפיון ובסיס מדעי
מסמך ייזום למימוש (SRS / PRD)

שם הסטודנט/ית	מספר ת.ז.
נתנאל קדושי (Netanel Kadoshi)	318829041
אור סמולרצ'יק (Or Smolarchik)	208635045
רחל ברודסקי (Rachel Brodsky)	212781835
יוסף יוסף (Yosef Yosef)	211364740
ליאל בביין (Liel Babayn)	323834994

פריט	פרטים
מנחה	נתנאל בן חמו
מוסד	HIT – מכון טכנולוגי חולון
תאריך	04/10/2026
GitHub	SafeCross
סרטון הדגמה ב-Youtube	[קישור YouTube – יתווסף]

תוכן עניינים

0. כללי	4
0.1 מבוא – המצב הקיים (הגדרת הבעיה)	4
0.2 הרציונל לפיתוח המערכת	4
0.3 מטרות הפרויקט	4
0.4 מושגים והגדרות	4
1. תיאור המערכת	6
1.1 אופי האפליקציה	6
1.2 משתמשי המערכת	6
1.3 תרחישי שימוש (Use Cases)	6
1.3.1 התחברות (כל המשתמשים)	6
1.3.2 תרחישי "אדמין"	6
1.3.3 תרחישי "מוקדן"	7
1.3.4 תרחישי "מנהל"	7
1.3.5 תרחישי "טכנאי"	7
1.3.6 תרחיש חיישן ה-AI	7
1.4 תרשים זרימת משתמשים (User Navigation Flow)	7
2. דרישות פונקציונליות (Functional Requirements)	8
3. דרישות לא פונקציונליות	9
3.1 ביצועים	9
3.2 אבטחת מידע	9
3.3 יכולת התרחבות	9
3.4 אמינות	9
3.5 שימושיות ותחזוקתיות	9
4. ארכיטקטורה ועיצוב המערכת	10
4.1 סקירת הארכיטקטורה	10
4.2 ארכיטקטורת צד שרת (מודולי השרת)	11
4.2.1 ממשק REST	11
4.2.2 אירועי זמן אמת (Socket.io)	12
4.3 ארכיטקטורת צד הלקוח	12
4.4 צינור שירות ה-AI	12
5. מבנה הנתונים (מסד הנתונים)	13
5.1 אוספים	13
6. בסיס מדעי, טכנולוגיות ואלגוריתמים	14
6.1 זיהוי אובייקטים – רשתות קונבולוציה (YOLOv8)	14
6.2 מעקב אחר אובייקטים מרובים	14
6.3 גאומטריה – מרחק משפת המדרכה	14
6.4 סיווג סיכון קינמטי (מערכת מומחה מבוססת כללים)	14
6.5 מדעי הנתונים – חישוב סטטיסטיקות בצד השרת	15
6.6 אבטחת מידע	15
6.7 תקשורת מונחית אירועים ובזמן אמת	15

6.8 מחסנית הטכנולוגיות	16
7. ממשק המשתמש (UI)	17
7.1 לוח האדמין – משתמשים ותשתיות חומרה	17
7.2 לוח המנהל – ניתוח בטיחות	17
7.3 לוח המוקדן – מוקד שליטה בזמן אמת	18
7.4 לוח הטכנאי – תחזוקה בשטח	18
8. תהליך העבודה (Work Flow)	19
9. בדיקות ואימות	20
9.1 בדיקות מקצה לקצה לשרת	20
9.2 בדיקות יחידה לשירות ה־AI	20
9.3 בדיקות ידניות ובדיקות אינטגרציה	20
10. סקירת הפתרון והמימוש	21
10.1 אתגרים מרכזיים ופתרונות	21
10.1.1 דרישות משתנות	21
10.1.2 סנכרון רב־תפקידי בזמן אמת	21
10.1.3 סטטיסטיקות אמינות	21
10.1.4 כללי AI שאינם תלויים במצלמה	21
10.1.5 הרחבת הסכמה בלי לשבור נתונים	21
11. המלצות להמשך פיתוח	23
קישורים	23

לחצו לחיצה ימנית על התוכן וב־"עדכן שדה" כדי לרענן את מספרי העמודים.

0. כללי

0.1 מבוא – המצב הקיים (הגדרת הבעיה)

מעברי חצייה וצמתים אחראים לחלק לא פרופורציונלי מתאונות הדרכים והכמעט-תאונות בעיר. משתמשי הדרך הפגיעים ביותר – ילדים, הולכי רגל המוסחים בטלפון ואנשים שחוצים במהירות או באופן בלתי צפוי – הם בדיוק אלה שהנהג מתקשה לצפות.

כיום ניטור מעברי חצייה מתבסס על שלוש גישות, ולכל אחת פער ברור:

- **מצלמות CCTV מסורתיות** – הקלטה פסיבית שמשמשת בעיקר לחקירה לאחר תאונה. איש אינו מקבל התראה בזמן שהמצב המסוכן מתפתח.
 - **מעברי חצייה פסיביים (ללא רמזור)** – תלויים לחלוטין בערנות הנהג. נהגים רבים אינם מבחינים בהולך רגל הממתין בשפת המדרכה, והולכי רגל עלולים לרדת לכביש באופן פתאומי.
 - **חיישני תנועה בסיסיים** – מגיבים לכל תנועה ולכן יוצרים שיעור גבוה של התראות שווא, עד שהמפעילים מתעלמים מהן.
- בנוסף, ניטור ידני אינו ברי-הרחבה: מפעיל אנושי אינו יכול לצפות ברציפות במוקדים רבים, ואין מקום אחד שבו נאספים אירועים, מצב הצידוד וסטטיסטיקות לצורך קבלת החלטות מבוססת נתונים.

0.2 הרציונל לפיתוח המערכת

SafeCross סוגרת את הפערים האלה בכך שהיא הופכת מצלמה רגילה במעבר חצייה לחיישן בטיחות פעיל:

1. **זיהוי בזמן אמת** של הולכי רגל, ילדים, כלי רכב וטלפונים באזור ההמתנה שלפני המעבר, באמצעות מודל למידה עמוקה לזיהוי אובייקטים (YOLOv8).
2. **סיווג סיכון התנהגותי** – לא כל חצייה היא התראה. כל הולך רגל במעקב מסווג לאחד מ-17 תרחישי סיכון מוגדרים (נמוך / בינוני / גבוה), כך שרק התנהגות בלתי צפויה או מסוכנת מפעילה התראה.
3. **התראות מיידיות** נשלחות בזמן אמת למוקד הבקרה (מוקדן), כולל תמונה של האירוע ומיקומו על המפה.
4. **אימות אנושי** – המוקדן מאשר ומטפל בכל התראה; רק התראות שטופלו נכנסות לסטטיסטיקות הניהוליות, וכך הנתונים אמינים.
5. **ניהול תפעולי** – תשתיות (צמתים, מצלמות, פסי LED להתרעה), משתמשים ומצב תחזוקה מנוהלים במערכת אחת, עם סנכרון חי בין כל המסכים המחוברים.

0.3 מטרות הפרויקט

- בניית פלטפורמת Web רב-תפקידית מקצה לקצה ובזמן אמת, המשלבת ראייה ממוחשבת (Python ו-YOLOv8), שרת REST ו-WebSocket (Node.js ו-Express) ומסד נתונים בענן (MongoDB Atlas).
- זיהוי וסיווג אוטומטי של התנהגות מסוכנת ובלתי צפויה של הולכי רגל והעברת התראות למפעילים בתוך שניות.
- מתן לוחות בקרה ייעודיים לארבעה תפקידים: אדמין, מנהל, מוקדן וטכנאי.
- חישוב סטטיסטיקות בטיחות בצד השרת (Aggregation) לתמיכה בהחלטות מבוססות נתונים של הרשויות.
- הכנת המערכת להפעלת שלט התרעה פיזי (LED) לנהגים – כל התראה נושאת דגל **ledTriggered**.

0.4 מושגים והגדרות

מונח	הגדרה
צומת / מעבר חצייה	מעבר חצייה מנוטר. נשמר עם שם, עיר, רחוב, קואורדינטות (lat/long), סטטוס ודגל "אזור מוסד חינוך".
אזור השפה (ROI)	מצולע (בקואורדינטות מנורמלות 0-1) המסמן את אזור המעבר / הכביש בפריים של המצלמה. המרחקים ממדדים מכפות הרגליים של האדם אל קו השפה של המצולע.

מונח	הגדרה
יחידת גובה גוף (h)	גובה המלבן התוחם של האדם. מרחקים ומהירויות מבוטאים ב־h/s, ולכן הכללים אינם תלויים במרחק המצלמה וברזולוציה.
מסלול (Track)	רצף זיהויים של אותו אדם לאורך פריימים, עם מזהה קבוע.
תרחיש סיכון	אחת מ־17 התנהגויות מוגדרות (למשל H1 "התפרצות לכביש", M2 "התקרבות תוך אחיזת טלפון"), שלכל אחת מותאמת רמת חומרה.
חומרה	נמוכה (תיעוד בלבד), בינונית (התרעה מוקדמת) או גבוהה (סכנה מיידית).
ledTriggered	דגל על התראה: true עבור התראות בינוניות/גבוהות, כלומר ההתראה אמורה להפעיל את ה־LED במעבר החצייה.
התראה שטופלה	התראה שמוקדן סימן כמטופלת (<code>isResolved = true</code> , ו־ <code>resolvedAt</code> נקבע על ידי השרת).
JWT	JSON Web Token – טוקן המוחזר בהתחברות ומזהה את המשתמש ואת תפקידו בכל בקשה.
מפתח API (x-api-key)	סוד משותף המזהה את שירות ה־AI (מכונה מול מכונה) במקום טוקן של משתמש.
Change Stream	יכולת של MongoDB להודיע לשרת על כל הוספה/עדכון באוסף; משמשת לשידור התראות בזמן אמת.

1. תיאור המערכת

1.1 אופי האפליקציה

- אפליקציית Web מסוג SPA הבנויה ב־React ו־Vite ומעוצבת ב־Tailwind CSS. הממשק רספונסיבי ועובד גם על מסכים במוקד הבקרה וגם על מכשירים ניידים בשטח.
- שרת Node.js ו־Express החושף REST API ושרת WebSocket של Socket.io על אותו פורט.
- שירות AI נפרד ב־Python שמנתח וידאו (באמצעות YOLOv8 + מעקב + כללי סיכון) ושולח התראות לשרת ב־HTTPS עם מפתח API.
- הנתונים נשמרים ב־MongoDB Atlas (NoSQL בענן) באמצעות סכמות Mongoose; תמונות האירועים נשמרות ב־Cloudinary.
- רכיב הזדהות (התחברות למערכת) מנפיק JWT; כל מסך וכל קריאת API מוגבלים לפי תפקיד המשתמש.
- המפות מוצגות באמצעות (React-Leaflet) Leaflet ו־OpenStreetMap; כתובות מומרות לקואורדינטות בעזרת שירות Nominatim.

1.2 משתמשי המערכת

תפקיד	אחריות	יכולות עיקריות
אדמין	מנהל המערכת הראשי	יצירה, עריכה, השעיה ומחיקה של משתמשים; יצירה, עריכה ומחיקה של צמתיים, מצלמות ופסי LED (עם המרת כתובת אוטומטית); רואה כל שינוי בזמן אמת.
מנהל	מעקב וניתוח ניהולי	צפייה בסטטיסטיקות המחשבות בשרת (מדדים, 5 הצמתיים המסוכנים, אזורי מוסדות חינוך, מגמה שבועית, פילוח חומרה); הלוח מתעדכן כשהתראות מסומנות ע"י המוקדן כמטופלות. בעל יכולת לייצא דוח PDF.
מוקדן	ניטור תפעולי בזמן אמת	טבלת התראות חיה ומפה אינטראקטיבית; סימון התראות כמטופלות / פתיחה מחדש; השינויים מסונכרנים לכל המוקדנים במשמרת.
טכנאי	תחזוקה בשטח	רואה את כל הציוד בסטטוס "תקלה" שסומן ע"י האדמין מגיע לצומת ומסמן את הציוד כתקין (פעיל).
חיישן AI (service)	משתמש מכונה	שולח התראות סכנה עם מפתח API; לעולם אינו משתמש בטוקן משתמש.

1.3 תרחישי שימוש (Use Cases)

1.3.1 התחברות (כל המשתמשים)

- המשתמש מזין שם משתמש וסיסמה. השרת מאמת מול ה־hash של bcrypt ומחזיר JWT (בתוקף 24 שעות) יחד עם התפקיד.
- הממשק מנתב את המשתמש ללוח המתאים: /admin, /manager, /dispatcher או /technician. משתמש מושהה מקבל הודעה שהחשבון הושהה.

1.3.2 תרחישי "אדמין"

- **ניהול משתמשים** – יצירת משתמש עם תפקיד, עדכון פרטים או תפקיד, השעיה / הפעלה מחדש ומחיקה. כל שינוי משודר (`user_added`, `user_updated`, `user_deleted`) לכל האדמינים המחוברים.
- **ניהול תשתיות חומרה** – הוספה או עריכה של צומת (שם, סטטוס, עיר, רחוב). הקואורדינטות מתמלאות אוטומטית מהכתובת. הוספת מצלמות (IP, סוג: PTZ / LPR / תרמית) ופסי LED (צבע) ושיוכם לצומת. סימון צומת כאזור מוסד חינוך.
- **סנכרון ניהולי חי** – אחרי כל יצירה/עדכון/מחיקה השרת משדר `infra_added` / `infra_updated` / `infra_deleted` עם `{ type, payload }`, כך שכל האדמינים רואים את אותם נתונים ללא רענון.

1.3.3 תרחישי "מוקדן"

- במסך הראשי המוקדן רואה מפה אינטראקטיבית של כל הצמתים וטבלת התראות חיה (חומרה, צומת, שעה, סוג הולך רגל, תמונה).
- התראה חדשה מופיעה מיד (אירוע `newAlert`) בראש הטבלה ללא רענון הדף.
- המוקדן בוחן את האירוע ולוחץ "טופל". השרת שומר `isResolved = true` ואת `resolvedAt`, ומשדר `alertUpdated` ו-`alert_resolved`. הסטטוס מתעדכן אצל כל שאר המוקדנים ומונע כפילות עבודה. ניתן לפתוח מחדש התראה שטופלה (`alert_reopened`).
- מהמפה ניתן לפתוח את תיק הצומת (`/crosswalk/:id`) עם היסטוריית האירועים ומצב הציוד.

1.3.4 תרחישי "מנהל"

- המנהל בוחר מסך: 5 המסוכנים ביותר, סביבת מוסדות חינוך, או כל הצמתים.
- הלוח מציג כרטיסי מדדים (סה"כ התראות שטופלו, התראות בסיכון גבוה, צמתים פעילים / סה"כ, זמן תגובה ממוצע), גרף עמודות לפי צומת (ילדים / מבוגרים / כלי רכב), גרף שבועי (ראשון-שבת) וגרף עוגה של חומרת האירועים.
- כאשר מוקדן כלשהו מטפל בהתראה, לוח המנהל מקבל `alert_resolved` ומושך מחדש את הסטטיסטיקות עבור המסך הפעיל.
- המנהל יכול לייצא את הלוח כדוח PDF באמצעות הכפתור "יצא דוח PDF".

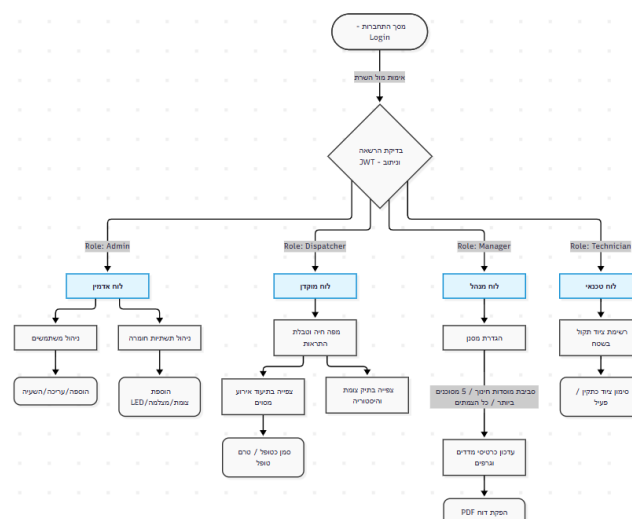
1.3.5 תרחישי "טכנאי"

- הטכנאי רואה רשימה של ציוד תקול – מצלמות, פסי LED וצמתים בסטטוס "תקלה" או "מושהה" – עם מיקום הצומת.
- לאחר התיקון הטכנאי מסמן את הפריט כתקין. הסטטוס משתנה ל"פעיל" (PUT על הציוד) והעדכון נדחף בזמן אמת לאדמינים, למנהלים ולמוקדנים (מקור אמת יחיד).

1.3.6 תרחיש חיישן ה-AI

- שירות ה-AI מנתח את וידאו המצלמה, מזהה התנהגות מסוכנת ושולח `POST /api/alerts` עם הכותרת `x-api-key`, החומרה, רמת הביטחון, סוג הולך הרגל, מרחק / מהירות ותמונה (אם קיימת).
- השרת מאמת את המפתח, מעלה את התמונה ל-Cloudinary, שומר את ההתראה ב-MongoDB, וה-Change Stream דוחף אותה למוקדנים.

1.4 תרשים זרימת משתמשים (User Navigation Flow)



איור 1 – ניתוב משתמשים ואפשרויות פעולה לפי הרשאת מערכת (Role-Based Navigation).

2. דרישות פונקציונליות (Functional Requirements)

עדיפות: M = חובה, S = רצוי.

מזהה	דרישה	תפקיד	עד.
FR-01	משתמשים מתחברים עם שם משתמש וסיסמה ומקבלים JWT ואת תפקידם.	כולם	M
FR-02	כל משתמש מנותב ללוח של תפקידו; לוחות ו-API אחרים חסומים (403).	כולם	M
FR-03	האדמין יכול ליצור, לעדכן, להשעות/להפעיל ולמחוק משתמשים. אין הרשמה פומבית.	אדמין	M
FR-04	האדמין יכול ליצור, לעדכן ולמחוק צמתים, מצלמות ופסי LED; מצלמות ופסי LED משויכים לצומת.	אדמין	M
FR-05	המערכת ממירה אוטומטית את כתובת הצומת לקואורדינטות (Geocoding).	אדמין	S
FR-06	כל שינוי בתשתיות או במשתמשים נדחף בזמן אמת לכל האדמינים המחוברים.	אדמין	M
FR-07	שירות ה-AI מזהה הולכי רגל, כלי רכב וטלפונים בכל פריים שנותח.	AI	M
FR-08	שירות ה-AI עוקב אחרי כל אדם בין פריימים ומסווג את ההתנהגות ל-17 תרחישי סיכון עם חומרה וציון ביטחון.	AI	M
FR-09	שירות ה-AI שולח לשרת התראה אחת לכל אירוע, מאומתת במפתח API, יחד עם תמונה של האירוע. אם העלאת התמונה נכשלת, ההתראה נשמרת גם בלעדיה.	AI	M
FR-10	התראות חדשות מופיעות בזמן אמת בלוח המוקדן, בטבלה ועל המפה.	מוקדן	M
FR-11	המוקדן יכול לסמן התראה כמטופלת או לפתוח אותה מחדש; השרת שומר את זמן הטיפול, המשמש לחישוב זמן התגובה הממוצע בלוח המנהל.	מוקדן	M
FR-12	לוח המנהל מציג סטטיסטיקות המחושבות בשרת עם מסנן 5 מסננים / מוסדות חינוך / הכל.	מנהל	M
FR-13	רק התראות שטופלו בספרות בסטטיסטיקות; הלוח מתעדכן כאשר התראה מטופלת.	מנהל	M
FR-14	הטכנאי רואה את כל הציוד בסטטוס תקלה/מושהה ויכול לסמן אותו כתקין.	טכנאי	M
FR-15	לכל צומת יש דף פרטים עם היסטוריית ההתראות ומצב הציוד.	מוקדן	S
FR-16	התראות בחומרה בינונית/גבוהה מסומנות להפעלת התרעת LED (ledTriggered).	AI / מערכת	S
FR-17	המנהל יכול לייצא את לוח הסטטיסטיקות כדוח PDF.	מנהל	S

3. דרישות לא פונקציונליות

3.1 ביצועים

- התראה מגיעה למסך המוקדן בתוך כשנייה מרגע שהשרת קיבל אותה (Change Stream של MongoDB ← דחיפה ב-Socket.io, ללא דגימה חוזרת).
- שירות ה-AI מנתח פריים אחד מכל N פריימים (ברירת מחדל N = 3) ברזולוציה מוקטנת (רוחב 960 פיקסלים) כדי לרוץ על מעבד רגיל; השליחה לרשת רצה ב-thread ברקע ואינה עוצרת את ניתוח הווידאו.
- הסטטיסטיקות מחושבות ב-Aggregation יחיד של MongoDB (\$facet) במקום טעינת כל ההתראות לזיכרון השרת.

3.2 אבטחת מידע

- סיסמאות נשמרות רק כ-hash של bcrypt; סיסמה גלויה לעולם אינה נשמרת או מוחזרת.
- כל בקשת משתמש נושאת `Authorization: Bearer <JWT>`; הטוקן חתום בסוד של השרת ופג תוקף לאחר 24 שעות.
- בקרת גישה מבוססת תפקידים (RBAC) בכל נתיב – למשל יצירת ציוד לאדמין בלבד, ועדכון ציוד לאדמין ולטכנאי.
- נתיבי מכונה (`POST /api/alerts`, `POST /api/detect`) מוגנים במפתח API המשווה בזמן קבוע (הגנה מפני מתקפות תזמון); השרת מסרב להפעילם ללא מפתח מוגדר (fail-closed).
- סודות (כתובת מסד הנתונים, סוד ה-JWT, מפתחות API, פרטי Cloudinary) נשמרים במשתני סביבה (env.) ואינם נשמרים ב-Git.
- שדות המנוהלים על ידי השרת (כגון `resolvedAt`) נדחים אם נשלחו על ידי הלקוח.

3.3 יכולת התרחבות

- REST API ללא מצב (stateless) – ניתן להוסיף מופעי שרת מאחורי מאזן עומסים; הזהות נמצאת ב-JWT.
- MongoDB Atlas הוא מסד נתונים מנוהל ומשוכפל בענן שניתן להגדילו אנכית או באמצעות sharding.
- שירות ה-AI הוא תהליך נפרד; ניתן להריץ מופע לכל מצלמה במקביל, וכולם מדווחים לאותו API.
- התמונות נשמרות ב-Cloudinary (CDN) ולא במסד הנתונים.

3.4 אמינות

- אם העלאת התמונה נכשלת, שירות ה-AI שולח את ההתראה מחדש ללא התמונה, כך שאירוע בטיחות לעולם אינו הולך לאיבוד.
- כשל בשידור ב-Socket אינו מכשיל את בקשת ה-HTTP – הכתיבה למסד הנתונים היא מקור האמת.
- התראות כפולות מצומצמות באמצעות חלון אירוע: התראה אחת לכל אירוע, עד שעוברות 3 שניות ללא אדם בסיכון.

3.5 שימושיות ותחזוקתיות

- ממשק רספונסיבי (Tailwind CSS) למסכי מוקד ולמכשירים ניידים; ממשק בעברית עבור המפעילים.
- קוד שכבתי ונקי: נתיבים ← middleware ← סרביסים ← מודלים, קובץ אחד לכל אחריות.
- בדיקות אוטומטיות: חבילת בדיקות מקצה לקצה לשרת ובדיקות יחידה לכללי ה-AI (ראו פרק 9).

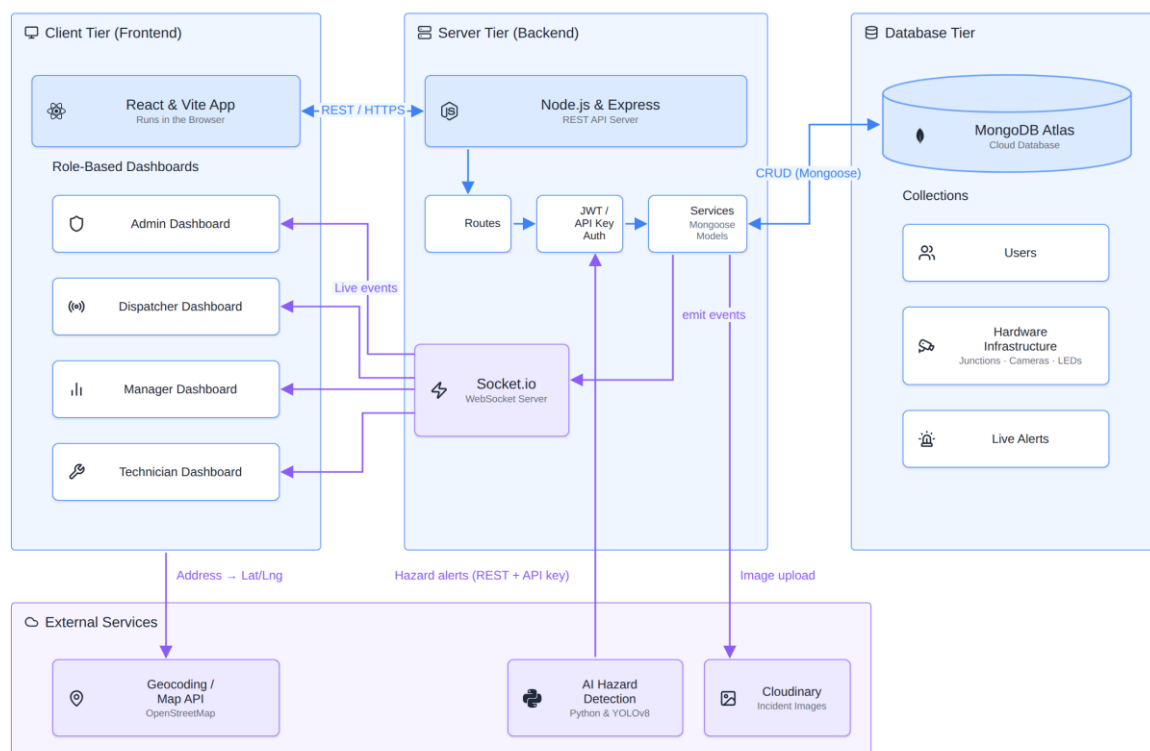
4. ארכיטקטורה ועיצוב המערכת

4.1 סקירת הארכיטקטורה

SafeCross בנויה כמערכת Web בשלוש שכבות, בתוספת שירותי AI חיצוני:

- **שכבת הלקוח** – SPA ב-React ו-Vite עם לוחות לפי תפקיד. מתקשרת עם השרת ב-REST (Axios/fetch) ומאזינה לאירועי Socket.io.
- **שכבת השרת** – REST API ב-Node.js ו-Express ושרת Socket.io. בקשות עוברות דרך נתיבים – אימות (JWT או מפתח API) – הרשאת תפקיד – שירותים (services) – מודלים של Mongoose.
- **שכבת הנתונים** – MongoDB Atlas עם חמישה אוספים: users, crosswalks, cameras, leds, alerts.
- **שירותים חיצוניים** – זיהוי סכנות ב-AI עם Python YOLOv8, Cloudinary (תמונות אירועים), OpenStreetMap / Nominatim (מפות והמרת כתובות, מופעלים מהלקוח).

SafeCross — Real-Time Smart Traffic Management System Architecture



איור 2 – ארכיטקטורת המערכת וזרימת הנתונים

4.2 ארכיטקטורת צד שרת (מודולי השרת)

מודול	קבצים	אחריות
נקודת כניסה	<code>index.js, app.js</code>	<code>app.js</code> בונה את אפליקציית Express (CORS, גוף JSON עד 10MB, נתיבים). <code>index.js</code> יוצר את שרת ה-HTTP, מחבר את Socket.io, מתחבר ל-MongoDB ומפעיל את ה-Change Streams.
נתיבים	<code>routes/*.js</code>	Router אחד לכל משאב: <code>alerts, crosswalks, cameras, leds, users, analytics, detect</code> .
Middleware	<code>middleware/*.js</code>	(JWT) <code>authenticationMiddleware</code> , <code>authorizationMiddleware</code> (תפקידים), <code>apiKeyMiddleware</code> (מפתח החיפוש).
שירותים	<code>services/*.js</code>	לוגיקה עסקית: CRUD לכל משאב, יצירת התראה עם העלאת תמונה, חישוב סטטיסטיקה, ניהול משתמשים.
מודלים	<code>models/*.js</code>	סכמות Mongoose עם ולידציה (שדות חובה, enums, הפניות).
תצורה	<code>config/db.js, socket.js, cloudinary.js</code>	חיבור למסד הנתונים, שרת Socket.io, פונקציות שידור ו-Change Streams, הגדרת Cloudinary.
שירות AI	<code>backend/ai-service/*.py</code>	<code>main.py</code> (וידאו), <code>detector.py</code> (YOLO), <code>tracker.py</code> , <code>risk_rules.py</code> , <code>alert_sender.py</code> , <code>app.py</code> (FastAPI/detect).

4.2.1 ממשק REST

שיטה ונתיב	הרשאה	תיאור
POST /api/users/login	פומבי	התחברות { token, role, username }
POST /api/users/register	אדמין	יצירת משתמש
GET / PUT / DELETE /api/users/:id	אדמין	רשימה, עדכון ומחיקה של משתמשים
PATCH /api/users/:id/status	אדמין	השעה / הפעלה של משתמש
GET /api/crosswalks · /cameras · /leds	כל התפקידים	רשימת כל הציוד (אפשר <code>?junctionId=</code>)
POST /api/crosswalks · /cameras leds/	אדמין	יצירת ציוד <code>infra_added</code>
PUT /api/crosswalks · /cameras leds/:id/	אדמין, טכנאי	עדכון ציוד <code>infra_updated</code>
DELETE /api/crosswalks · /cameras leds/:id/	אדמין	מחיקת ציוד <code>infra_deleted</code>
POST /api/alerts	חיישן (x-api-key)	יצירת התראה (העלאת תמונה ל-Cloudinary)
GET /api/alerts	כל התפקידים	רשימת התראות (החדשות ראשונות)
PUT /api/alerts/:id	אדמין, מוקדן, טכנאי	סימון כמטופלת / פתיחה מחדש <code>alert_resolved / alert_reopened</code>
GET /api/analytics/dashboard?filter	מנהל, אדמין	סטטיסטיקות עבור <code>top5 / school / all</code>
POST /api/detect	חיישן (x-api-key)	העברת תמונה אחת לשירות ה-AI לזיהוי

4.2.2 אירועי זמן אמת (Socket.io)

אירוע	מתי משודר	מאזינים
newAlert	נוספה התראה חדשה (Change Stream)	מוקדן, פרטי הצומת
alertUpdated	מסמך התראה השתנה (Change Stream)	מוקדן, פרטי הצומת
alert_resolved / alert_reopened	מוקדן מטפל בהתראה / פותח אותה מחדש	מנהל (משיכת סטטיסטיקות מחדש)
/ infra_added / infra_updated / infra_deleted	ציוד נוצר / עודכן / נמחק (מטען { type, payload })	אדמין, מנהל, טכנאי
/ user_added / user_updated / user_deleted	משתמש השתנה (Change Stream על users)	אדמין

4.3 ארכיטקטורת צד הלקוח

אפליקציית הלקוח היא Single Page Application (SPA) הבנויה ב-React ו-Vite. היא מספקת חווית משתמש חלקה ומהירה, ללא טעינה מחדש של הדף. צד הלקוח מבוסס על המנגנונים הבאים:

- **ניהול מצב ומחזור חיים:** (useEffect, useState) React Hooks מנהלים את הנתונים המקומיים של כל מסך. החיבור לשרת Socket.io מעביר אירועים בזמן אמת שמעדכנים ישירות את מצב הרכיבים, כך ש-React מרנדר מחדש רק את החלק שהשתנה בדף.
- **ניווט לפי תפקיד:** לאחר ההתחברות השרת מחזיר JWT ואת תפקיד המשתמש. הלקוח שומר את הטוקן ב-`localStorage`, ו-React Router מעביר את המשתמש ללוח המתאים. בהתנתקות הטוקן נמחק. בקרת הגישה עצמה נאכפת בשרת, שדוחה כל בקשה ללא טוקן תקף או עם תפקיד שאינו מורשה (403 / 401).
- **תקשורת מאובטחת מול השרת:** כל קריאת REST (Fetch API) נושאת את ה-JWT בותרת `Authorization: Bearer`. כתובת השרת נקראת ממשתנה סביבה (`VITE_API_URL`), כך שאותו קוד רץ גם מקומית וגם בסביבת הייצור.
- **מבנה הדפים ועיצוב רספונסיבי:** לכל תפקיד יש דף לוח בקרה משלו. העיצוב נעשה ב-Tailwind CSS עם נקודות שבירה רספונסיביות, כך שהממשק מתאים גם למסכים גדולים במוקד השליטה וגם למכשירים ניידים של טכנאי השטח. המפות מוצגות באמצעות React-Leaflet והגרפים באמצעות Recharts.

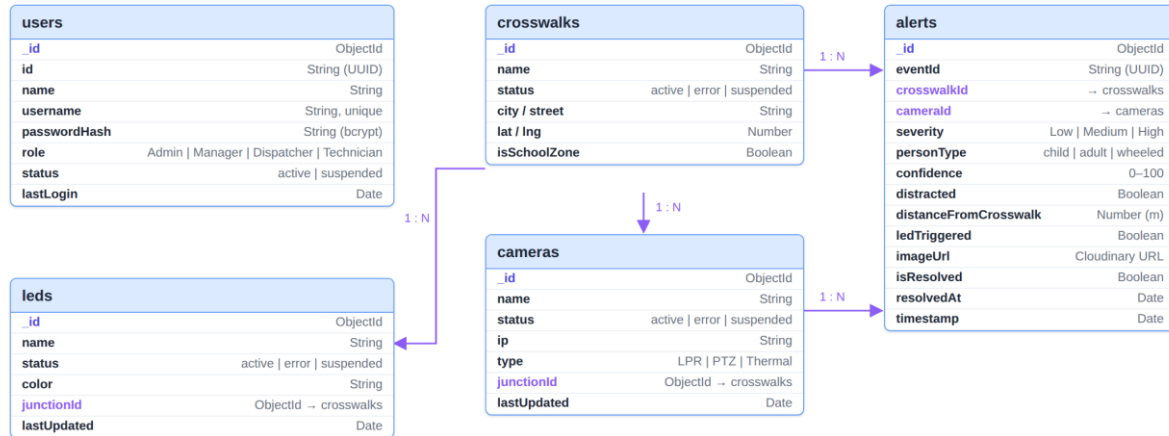
4.4 צינור שירות ה-AI

`main.py` קורא קובץ וידאו או זרם מצלמה ומריץ על כל פריים שנבחר: `detector` (YOLOv8) ← הולכי רגל, כלי רכב, אופניים/אופנועים, טלפונים) ← `tracker` (מזהה קבוע לכל אדם) ← `risk rules` (מאפייני תנועה ו-17 תרחישים) ← `alert sender` (תור queue ברקע ← `POST /api/alerts`). בנוסף, אפליקציית FastAPI (`app.py`) חושפת `POST /detect` לזיהוי בתמונה בודדת, שבה משתמש הנתב `api/detect/` בשרת.

5. מבנה הנתונים (מסד הנתונים)

מסד הנתונים הוא MongoDB Atlas (NoSQL מבוסס מסמכים). כל אוסף מוגדר בסכמת Mongoose האוכפת טיפוסים, שדות חובה וערכים מותרים (enums). מצלמות ופסי LED מפנים לצומת שלהם באמצעות ObjectId (junctionId); התראות שומרות את מזהי הצומת והמצלמה.

SafeCross — MongoDB Collections (Mongoose schemas)



איור 3 – האוספים והקשרים ביניהם

5.1 אוספים

אוסף	מטרה	שדות מרכזיים
users	חשבונות והרשאות	username (יחודי), passwordHash (bcrypt), role, status, lastLogin
crosswalks	צמתים מנוטרים	name, status, city, street, lat, lng, isSchoolZone
cameras	מצלמות בצומת	name, status, ip, type, junctionId
leds	פסי LED להתרעה בצומת	name, status, color, junctionId
alerts	אירועי סכנה מה-AI	crosswalkId, cameraId, severity, personType, confidence, distracted, distanceFromCrosswalk, approachSpeed, ledTriggered, imageUrl, isResolved, resolvedAt, timestamp

טיפוס הנתונים של כל שדה מופיע בתרשים האוספים שלמעלה (איור 3).

סטטוס הצידוד משתמש ברשימה משותפת אחת – `active` | `error` | `suspended` – כך שמסך הטכנאי מטפל בכל החומרה באותה דרך.

6. בסיס מדעי, טכנולוגיות ואלגוריתמים

הפרויקט מיישם חומר מקורסים באלגוריתמים, מבני נתונים, ראייה ממוחשבת ולמידת מכונה, מסדי נתונים, מדעי הנתונים, רשתות תקשורת ואבטחת מידע.

6.1 זיהוי אובייקטים – רשתות קונבולוציה (YOLOv8)

זיהוי האובייקטים מבוצע באמצעות YOLOv8 (Ultralytics), רשת נוירונים קונבולוציונית חד-שלבית. בשונה ממזהים דו-שלביים (כגון Faster R-CNN), חוזה את המלבנים התוחמים ואת הסתברויות המחלקות לכל התמונה במעבר יחיד ברשת, ולכן מתאים לוידאו בזמן אמת. הרשת אומנה מראש על מאגר COCO. SafeCross משתמשת רק במחלקות הרלוונטיות: אדם (0), אופניים (1), מכונית (2), אופנוע (3), אוטובוס (5), משאית (7) וטלפון נייד (67).

- **סף ביטחון** – זיהויים מתחת ל-0.5 נזרקים כדי להפחית התראות שווא.
- **Non-Maximum Suppression** ובדיקת IoU נוספת (חפיפה מעל 0.7) מסירים מלבנים כפולים של אותו אדם: $IoU(A,B) = \text{area}(A \cap B) / \text{area}(A \cup B)$.
- **בחירת המודל** – **yolo8n (nano)** נותן את האיזון הטוב ביותר בין מהירות לדיוק על מעבד רגיל; ניתן להחליפו במודל גדול יותר לדיוק גבוה יותר.

6.2 מעקב אחר אובייקטים מרובים

המזהה רואה כל פריים בנפרד, אבל התנהגות (מהירות, כיוון, עצירה) ניתנת למדידה רק לאורך זמן. רכיב המעקב מקשר זיהויים למסלולים עם מזהה קבוע. קיימים שני מימושים חלופיים:

- **SimpleTracker** (ברירת מחדל) – התאמה חמדנית לשכן הקרוב עם שער (gating): זיהוי מותאם למסלול קיים רק אם תנועתו לשנייה מתחת לגבול סביר (6 גבהי גוף לשנייה, לפחות 20 פיקסלים) ויחס גובה המלבן נשאר בטווח 0.6–1.7. מסלולים שנעלמים יותר מ-0.5 שניות נסגרים.
 - **ByteTrack** (Ultralytics) – אלגוריתם שיוך שמשתמש גם בזיהויים בעלי ביטחון נמוך כדי לשמור על מסלולים בזמן הסתרות.
- טלפונים משויכים לאדם הקרוב ביותר כאשר מרכז הטלפון קרוב לנקודת החזה שלו; אדם נחשב "מוסח" אם נראה טלפון בלפחות 50% מהפריימים האחרונים.

6.3 גאומטריה – מרחק משפת המדרכה

אזור המעבר מוגדר כמצולע בקואורדינטות מנורמלות, כך שאותה הגדרה עובדת בכל רזולוציה. עבור כל אדם, נקודת כפות הרגליים (מרכז תחתית המלבן) נבדקת באלגוריתם **נקודה בתוך מצולע** (`cv2.pointPolygonTest`), ומחושב **המרחק מנקודה לקטע** אל קווי השפה. התוצאה היא מרחק מסומן: חיובי על המדרכה, אפס או שלילי בתוך אזור המעבר.

כדי לנטרל את השפעת הפרספקטיבה (אנשים רחוקים נראים קטנים יותר), כל המרחקים והמהירויות מנורמלים לפי גובה הגוף של האדם עצמו, h . ילד מזוהה כאשר h קטן מ-0.9 מגובה המבוגר הייחוס באותו מיקום בתמונה.

6.4 סיווג סיכון קינמטי (מערכת מומחה מבוססת כללים)

לכל מסלול השירות מחשב מאפייני תנועה על חלון נע של 2 שניות: מרחק מהשפה, מהירות התקרבות (שינוי המרחק לאורך זמן), מהירות רוחבית, מהירות כוללת, תזוזה נטו, זמן עד השפה (מרחק ÷ מהירות התקרבות) ומהירות אחרונה ב-0.5 השניות האחרונות (כך שאדם שעצר כבר אינו התראה). המאפיינים מותאמים מול 17 כללים; כאשר כמה כללים מתאימים, רשימת עדיפויות בוחרת את החמור ביותר.

רמה	תרחישים (דוגמאות)	פעולה
נמוכה	L1 תנועה במקביל לכביש · L2 עמידה רחוק מהשפה · L3 התרחקות · L4 המתנה שקטה בשפה · M1 התקרבות רגילה · M3 קבוצה מתקרבת · M4 עמידה באזור המעבר	תיעוד בלבד
בינונית	M2 התקרבות תוך אחיזת טלפון · M5 התקרבות מהירה ללא האטה · H2 ילד מתקרב לשפה · MW כלי גלגלים מאט אך לא מספיק	התרעה מוקדמת, דגל LED
גבוהה	H1 התפרצות לכביש · H3 מוסח שאינו עוצר · H4 הופעה פתאומית ליד השפה · H5 כניסה לכביש ללא עצירה · H6 ילד מתפרץ לכביש · HW כלי גלגלים מתקרב במהירות	סכנה מיידית, דגל LED

הפירוט המלא של 17 הכללים והתנאים שלהם מופיע ב**בנספח א'**.

ציון הביטחון של התראה משלב את ביטחון המזהה עם כמות הראיות: $\text{confidence} = 100 \times \text{מוצע ביטחון הזיהוי} \times \text{חומרה}$ (1, מספר הפריימים / 3). אדם שנראה רק בפריים אחד או שניים מקבל ציון נמוך יותר. התראה נשלחת רק אחרי שהאדם נראה לפחות ב-3 פריימים שנותחו, ונשלחת התראה אחת לכל אירוע (אירוע מסתיים לאחר 3 שניות ללא אדם בסיכון).

6.5 מדעי הנתונים – חישוב סטטיסטיקות בצד השרת

הסטטיסטיקות של המנהל מחושבות בתוך MongoDB באמצעות **Aggregation Pipeline**. תחילה ההתראות מסוננות (`match`): רק התראות שטופלו בצמתים שנבחרו; לאחר מכן שלב `facet` יחיד מחשב ארבע תוצאות בלתי תלויות במעבר אחד על הנתונים:

- מדדים** – ספירה, מספר התראות בחומרה גבוהה וזמן תגובה ממוצע, `avg(resolvedAt - timestamp)` בדקות.
- לפי צומת** – `group` לפי צומת עם סכומים מותנים לילדים, למבוגרים ולכלי רכב.
- מגמה שבועית** – 7 הימים האחרונים (לפי שעון ישראל, עם `dateTrunc`) מקובצים לפי `dayOfWeek`, ומוחזרים כ-7 ערכים בדיוק מראשון עד שבת (ימים חסרים מקבלים 0).
- פילוח חומרה** – `group` לפי חומרה.

המסנן **5 המסוכנים ביותר** הוא שאילתת `top-k`: קיבוץ ההתראות שטופלו לפי צומת, מיון יורד לפי כמות ובחירת החמישה הראשונים. נכונות ארבע התוצאות נבדקות בבדיקות באמצעות חישוב בלתי תלוי מחדש ב-JavaScript.

6.6 אבטחת מידע

- גיבוב סיסמאות** – `bcrypt`, פונקציית גיבוב אדפטיבית עם `salt` (פקטור עלות 10), איטית בכוונה כהגנה מפני ניחוש בכוח.
- JWT** – תוכן הטוקן (מזהה משתמש, תפקיד) חתום ב-HMAC-SHA256; כל שינוי בטוקן פוסל את החתימה.
- השוואה בזמן קבוע** של מפתח ה-`(crypto.timingSafeEqual)` API מונעת מתקפות ערוץ צדדי מבוססות תזמון.

6.7 תקשורת מונחית אירועים ובזמן אמת

SafeCross פועלת לפי תבנית **Publish/Subscribe**. מסד הנתונים הוא מקור האמת היחיד: `Change Streams` של MongoDB (הדורשים `Replica Set`, כפי שמספק Atlas) מודיעים לשרת על כל שינוי, ו-Socket.io משדר את האירוע בחיבור WebSocket קבוע לכל הלקוחות הרשומים. כך אין צורך בדגימה חוזרת, ההשגה נמוכה וכל המסכים נשארים עקביים.

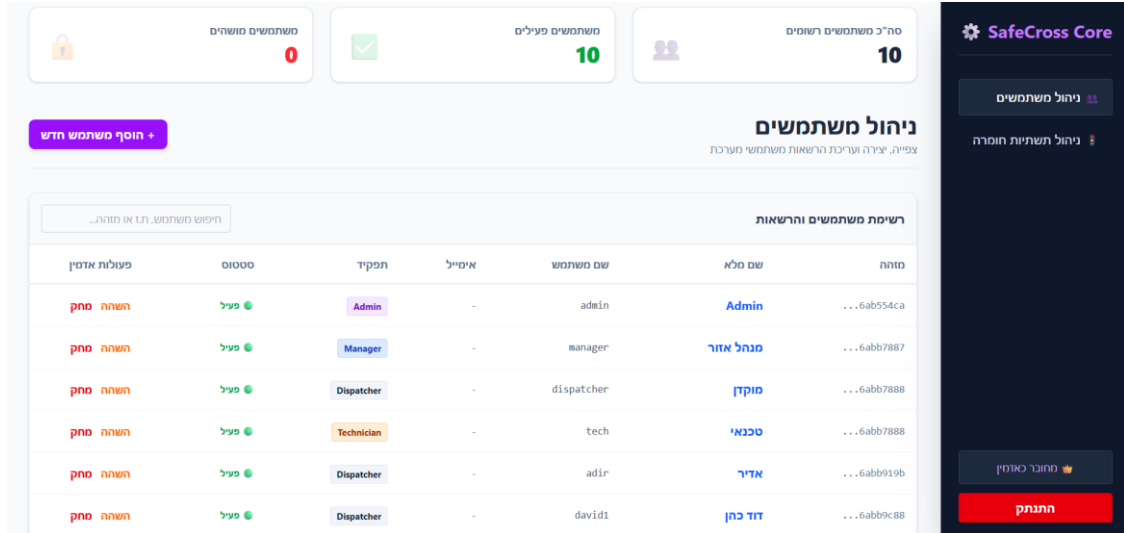
6.8 מחסנית הטכנולוגיות

שכבה	טכנולוגיות
לקוח	React 19 + Vite, React Router, Tailwind CSS, Axios, Socket.io client, Recharts (charts), Leaflet / React-Leaflet (maps)
שרת	Node.js, Express 5, Socket.io, Mongoose, jsonwebtoken, bcrypt, Cloudinary SDK, dotenv
נתונים ואחסון	MongoDB Atlas, Cloudinary
שירות AI	Python, Ultralytics YOLOv8, OpenCV, NumPy, FastAPI, Requests
שירותים חיצוניים	OpenStreetMap tiles, Nominatim geocoding
כלים	,Git & GitHub, VS Code, Postman, Node test runner, mongodb-memory-server Python unittest

7. ממשק המשתמש (UI)

הממשק בעברית, רספונסיבי ובעל מבנה אחיד: סרגל ניווט צדדי עם המיתוג של SafeCross ואזור עבודה מרכזי. כל תפקיד רואה רק את הלוח שלו.

7.1 לוח האדמין – משתמשים ותשתיות חומרה



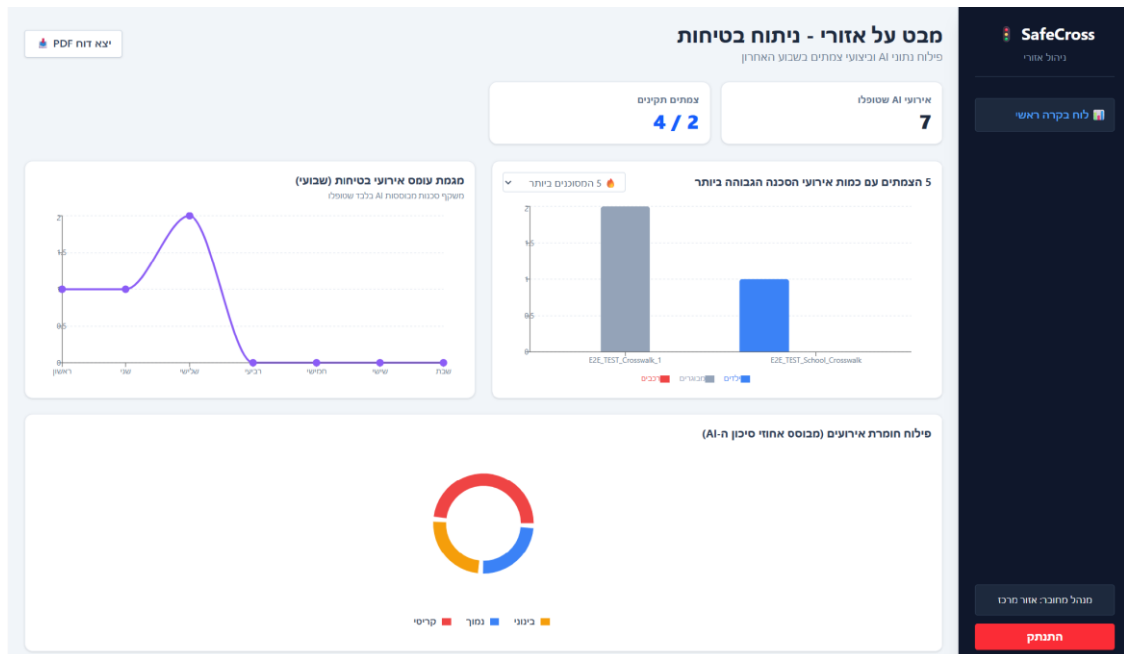
The screenshot shows the 'SafeCross Core' admin interface. At the top, there are three summary cards: 'משתמשים מושהים' (0), 'משתמשים פעילים' (10), and 'סה"כ משתמשים רשומים' (10). Below these is a 'ניהול משתמשים' (Manage Users) section with a table of users. The table has columns for 'מזהה' (ID), 'שם מלא' (Full Name), 'שם משתמש' (Username), 'אימייל' (Email), 'תפקיד' (Role), 'סטטוס' (Status), and 'פעולות אדמין' (Admin Actions). The roles listed are Admin, Manager, Dispatcher, and Technician. On the right sidebar, there are buttons for 'ניהול משתמשים' and 'ניהול תשתיות חומרה'.

מזהה	שם מלא	שם משתמש	אימייל	תפקיד	סטטוס	פעולות אדמין
...6abb554ca	Admin	admin	-	Admin	פעיל	השהה מחק
...6abb7887	מנהל אזור	manager	-	Manager	פעיל	השהה מחק
...6abb7888	מוקדן	dispatcher	-	Dispatcher	פעיל	השהה מחק
...6abb7888	טכנאי	tech	-	Technician	פעיל	השהה מחק
...6abb919b	אדיר	adir	-	Dispatcher	פעיל	השהה מחק
...6abb9c88	דוד כהן	davidi	-	Dispatcher	פעיל	השהה מחק

איור 4 – לוח האדמין

שני אזורי ניהול: ניהול משתמשים (יצירה, שינוי תפקיד, השעיה, מחיקה) ותשתיות חומרה (צמתים, מצלמות, פסי LED). כאשר מוזנת כתובת של צומת, הקואורדינטות מתמלאות אוטומטית.

7.2 לוח המנהל – ניתוח בטיחות



The screenshot shows the 'SafeCross' manager interface. It features a 'מבט על אזורי - ניתוח בטיחות' (Area Overview - Safety Analysis) section. This section includes a line graph showing 'מבט עומס אירועי בטיחות (שבועי)' (Weekly Safety Incident Load Overview) and a bar chart showing '5 הצמתים עם כמות אירועי הסכנה הגבוהה ביותר' (5 Intersections with the highest number of safety incidents). Below these is a donut chart showing 'פילוח חומרת אירועים (מבוסס אחוזי סיכון ה-)' (Incident Severity Breakdown (Based on H-Risk Percentages)). The right sidebar contains buttons for 'ניהול אזורי' and 'לוח בקרה ראשי'.

איור 5 – לוח המנהל

כרטיסי מדדים, גרף עמודות לפי צומת עם מסנן 5 מסוכנים / מוסדות חינוך / הכל, גרף מגמה שבועי וגרף עוגה של חומרת האירועים, כפתור "יצא דוח PDF" בראש המסך מאפשר למנהל לשמור או להדפיס את התצוגה הנוכחית.

7.3 לוח המוקדן – מוקד שליטה בזמן אמת

מוקד שליטה בזמן אמת
מנטר אזורים פעילים ומציג התרעות מבוססות חצלמה

מפת צמתים

חיסור API חי

פריסת צמתים (DB)

חפשו לפי רחוב, אזור או מזהה...

הרצל 45, חולון
מרכז העיר

אבן גבירול 10, תל אביב
צפון תל אביב

78 מסמכים dd/mm/yyyy סינון

תאריך ושעה	חומרה	תיאור אירוע	ניתוח AI (פילוח)	מיקום (מאקרו ומיקרו)	ציד מנטר	סטטוס טיפול	תיעוד
24.9.2026 19:43	בינוי	H2 - ילד מתקרב לכביש	69% כיסחון דיווי: child	0	e2e_cam e2e_test	סופל	צפה
24.9.2026 19:43	מונר	M4 - נכוחות באזור המעבר	91% כיסחון דיווי: מבוכר	0	e2e_cam e2e_test	סופל	אין

תצוגת מוקדן

סטטוס מערכת תקין

התנתק

איור 6 – לוח המוקדן

מפה אינטראקטיבית של הצמתים וטבלת התראות חיה עם תגיות חומרה ומתג "טופל / טרם טופל".

7.4 לוח הטכנאי – תחזוקה בשטח

ניהול תקלות חומרה
רשימת יצור קשר התרעות באמצע

רשימת תקלות בשימוש

סידור עבודה - רכיבים לחיפוש

סוג רכיב / מיקום	סוגה מערכת	חשור תקלה	מסלול סכנאי
צומת: כנולסון 70, חיפה	8ab0c0804c2b0acac2b94a7	תקלה מערכת מרכזית	מסלול סכנאי
מצלמה (IP: 127.0.0.10)	8ab0c0804c2b0acac2b94a7	תקלה במצלמה (אין)	מסלול סכנאי
תאורת LED (אין)	8ab0c0804c2b0acac2b94a7	תקלה תאורת	מסלול סכנאי

קראת שירות

סטטוס סכנאי

התנתק

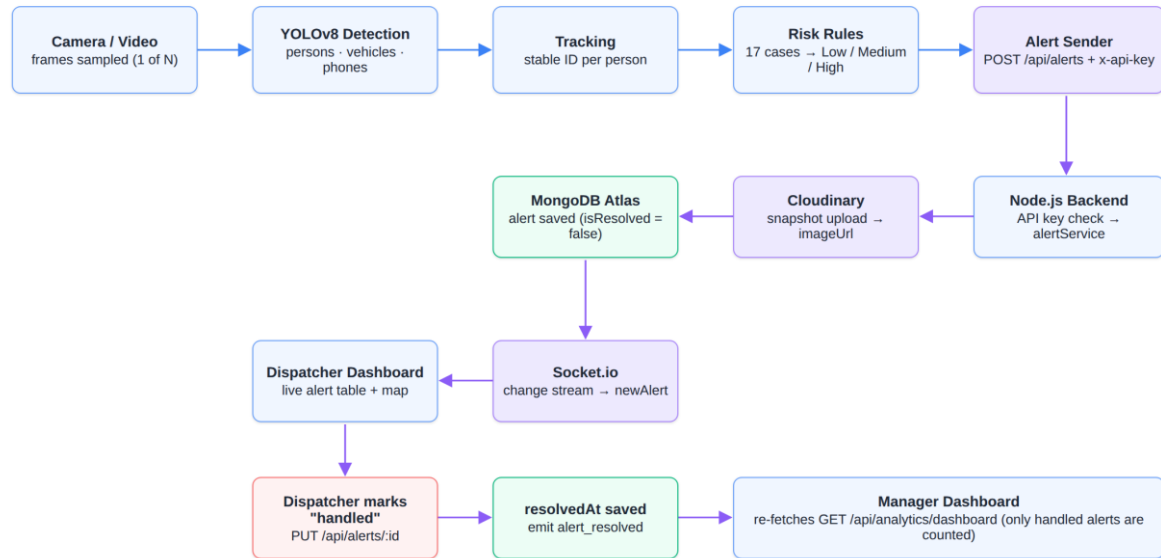
איור 7 – לוח הטכנאי

רשימת צידוד תקול לפי צומת, עם פעולה לסימון כל פריט בתקין.

8. תהליך העבודה (Work Flow)

מחזור החיים המלא של התראת סכנה, מהמצלמה ועד לסטטיסטיקות הניהוליות:

SafeCross — Hazard Alert Lifecycle



איור 8 – מחזור החיים של התראת סכנה

1. שירות ה-AI דוגם פריימים, מזהה אובייקטים עם YOLOv8 ועוקב אחר כל אדם.
2. כללי הסיכון מסווגים את ההתנהגות; תרחיש בינוני/גבוה יוצר התראה עם `ledTriggered = true`.
3. ההתראה נשלחת ל-`POST /api/alerts` עם מפתח ה-API; השרת מעלה את התמונה ל-Cloudinary ושומר את ההתראה.
4. ה-Change Stream דוחף `newAlert` לכל המוקדנים; ההתראה מופיעה בטבלה ועל המפה.
5. מוקדן מסמן את ההתראה כמטופלת; השרת שומר `resolvedAt` ומשדר `alertUpdated` ו-`alert_resolved`.
6. לוח המנהל מושך מחדש את הסטטיסטיקות, שכוללות כעת את ההתראה שטופלה.

9. בדיקות ואימות

9.1 בדיקות מקצה לקצה לשרת

חבילת בדיקות אוטומטית (Node test runner, `backend/tests/e2e.test.js`) מפעילה Replica Set אמיתי של MongoDB בזיכרון, מריצה את סקריפט ה-`seed` האמיתי, מפעילה את שרת Express ו-`Socket.io` האמיתי על פורט אקראי ומתחברת עם משתמשים אמיתיים. שום רכיב אינו מדומה. 24 הבדיקות מכסות:

- **Seed והתחברות** – הסבת נתונים ישנים, `seed` שאינו יוצר כפילויות, התחברות של ארבעת משתמשי ברירת המחדל עם התפקיד הנכון.
 - **תשתיות האדמין** – שליפת כל הציוד, יצירה / עדכון עם ולידציה (400), הרשאות (401 / 403), מזהה לא קיים (404), ואירועי `Socket` `infra_added` / `infra_updated` עם הסוג הנכון.
 - **מפתח API של החיישן** – בקשות ללא מפתח או עם מפתח שגוי נדחות (401); מפתח תקין שומר את ההתראה ומשדר `newAlert`.
 - **התראות** – `resolvedAt` נקבע על ידי השרת, ערך שנשלח מהלקוח נדחה, ו-`alert_resolved` משודר פעם אחת בדיוק בכל טיפול.
 - **סטטיסטיקות המנהל** – מבנה התשובה (7 ימים קבועים, 3 תוויות חומרה) וכל מסנן (all / school / top5) מושווים לחישוב בלתי תלוי ב-JavaScript מתוך מסד הנתונים; התראות שלא טופלו אינן נספרות.
- תוצאה: 24 מתוך 24 בדיקות עברו.

9.2 בדיקות יחידה לשירות ה-AI

בדיקות יחידה ב-`backend/ai-service/tests` (Python מאמתות את כללי הסיכון (41 בדיקות), את רכיב המעקב (11 בדיקות) ואת מעבד הווידאו (7 בדיקות) באמצעות מסלולים סינתטיים ווידאו שנוצר לצורך הבדיקה, כולל מקרי קצה כמו מלבנים חתוכים בשולי הפריים, קבוצות, טלפונים וכלי גלגלים.

9.3 בדיקות ידניות ובדיקות אינטגרציה

- כל נתיבי ה-REST נבדקו ב-Postman במהלך הפיתוח (קודי סטטוס, הרשאות ותוכן התשובות).
- התהליך המלא נבדק עם סרטוני הדוגמה: וידאו ← התראות AI ← מסך המוקדן ← "טופל" ← סטטיסטיקות המנהל.
- הסנכרון בין לקוחות מרובים נבדק בפתיחת כמה דפדפנים עם תפקידים שונים במקביל.

10. סקירת הפתרון והמימוש

הפרויקט נוהל במתודולוגיה אגילית בספרינטים חודשיים, עם תיאום דרך GitHub (ענפי שרת ולקוח נפרדים שמזוגו ל-main) ופגישות צוות קבועות.

ספרינט	מיקוד	תוצרים עיקריים
1	ארכיטקטורה ותכנון	דרישות, מבני נתונים, ארכיטקטורת המערכת, סביבות פיתוח (Node.js, Express, MongoDB Atlas, React & Vite, Tailwind CSS).
2	API, מסד נתונים ואבטחה	סכמות Mongoose, נתיבי REST, אימות JWT ו-middleware לתפקידים, נתוני דמו לפיתוח מקביל של הלקוח, בדיקות ב-Postman.
3	שילוב AI וממשק	סביבת Python עם YOLOv8, זיהוי הולכי רגל וכלי רכב באזור המעבר, לוחות לפי תפקיד.
4	סנכרון בזמן אמת וענן	מעקב בווידאו ו-17 תרחישי סיכון, סנכרון חי ב-Socket.io, תמונות ב-Cloudinary, סטטיסטיקות בשרת, מפתח API לחיפוש, בדיקות מקצה לקצה.

10.1 אתגרים מרכזיים ופתרונות

10.1.1 דרישות משתנות

האתגר: הדרישות השתנו תוך כדי בניית המערכת. צוות הלקוח הגדיר שדות, אירועים והרשאות חדשים לכל לוח, והמבנה הקיים בשרת כבר לא התאים. לדוגמה, עמוד האדמין דרש מבנה שונה לצמתים, למצלמות ולפסי ה-LED מזה שבבנה בספרינט 2.

הפתרון: עבדנו באיטרציות אגיליות, לוח אחד בכל פעם, עם פגישות תיאום בין צוותי הלקוח והשרת. כל חוזה API סוכם בכתב לפני הכתיבה: השדות, אירועי ה-Socket וההרשאות לכל תפקיד.

10.1.2 סנכרון רב-תפקידי בזמן אמת

האתגר: ארבעה תפקידים שונים עובדים על אותם נתונים באותו זמן, וכל אחד צריך לראות מיד שינויים אחרים. ללא תכנון ברור, מסכים הציגו נתונים לא מעודכנים, או ששני מוקדנים טיפלו באותה התראה.

הפתרון: קבענו שמסד הנתונים הוא מקור האמת היחיד. Change Streams של MongoDB ושידורים מהשרת מפיצים כל שינוי דרך Socket.io, לאחר שמיפנו איזה תפקיד צריך איזה אירוע: `newAlert` ו-`alertUpdated` למוקדנים, `alert_resolved` למנהלים, ואירועי `*_infra` ו-`*_user` לאדמינים ולטכנאים.

10.1.3 סטטיסטיקות אמינות

האתגר: ה-AI עלול לייצר התראות שווא. אילו כל התראה הייתה נכנסת ישירות לגרפים של המנהל, הסטטיסטיקות היו מטעות וההחלטות היו מתבססות על נתונים שגויים.

הפתרון: נספרות רק התראות שמוקדן אימת וסימן כמטופלות. כל החישובים רצים בשרת ב-Aggregation יחיד של MongoDB, כך שכל המנהלים רואים את אותם מספרים, והשרת עצמו רושם את זמן הטיפול המשמש לחישוב זמן התגובה הממוצע.

10.1.4 כללי AI שאינם תלויים במצלמה

האתגר: אדם רחוק מהמצלמה נראה קטן ונדמה שהוא נע לאט, ואילו אדם קרוב נראה גדול ומהיר. כללים שנכתבו בפיסקלים עבדו בסרטון אחד ונכשלו בזווית צילום או ברזולוציה אחרת.

הפתרון: כל המרחקים והמהירויות מנורמלים לפי גובה הגוף של האדם עצמו, ואזור המעבר מוגדר כמצולע בקואורדינטות מנורמלות (0-1). כך אותם 17 כללים עובדים בכל מיקום מצלמה ובכל רזולוציה.

10.1.5 הרחבת הסכמה בלי לשבור נתונים

האתגר: כאשר מבנה הנתונים השתנה, במסד הנתונים כבר היו מסמכים בפורמט הישן. מסמכים ישנים וחדשים יחד גרמו לשגיאות ב-API ולתוצאות שגויות במסכים.

הפתרון: סקריפט ה-seed כולל הסבה חד-פעמית: הוא מזהה מסמכים בפורמט הישן, מנקה אותם ומכניס את נתוני הדמו בפורמט החדש. בדיקות מקצה לקצה מתחילות מנתונים בפורמט הישן ומוודאות שההסבה עובדת ושהרצה חוזרת אינה משנה דבר.

11. המלצות להמשך פיתוח

- שליטה בשלט LED / התרעה פיזי – חיבור הדגל **ledTriggered** לבקר חומרה (למשל MQTT או נקודת REST בשלט) כדי להזהיר נהגים בזמן אמת.
- למידה מהמפעילים – שימוש בהחלטות המוקדנים ("טופל" / התראת שווא) כנתונים מתויגים לכוונן המודל וספי הכללים.
- זרמי מצלמה חיים – מעבר מקובצי וידאו לזרמי RTSP עם הסקה על GPU ותהליך AI לכל מצלמה.
- זיהוי אוטומטי של אזורי מוסדות חינוך – סימון אוטומטי של צמתים ברדיוס 300 מטר ממוסד חינוך, באמצעות אינדקס גאוגרפי על הקואורדינטות.
- יומן פעולות ודוחות – תיעוד כל פעולה והפקת דוחות תקופתיים לכל צומת עבור הרשויות.
- שילוב בעיר חכמה – חיבור לבקרי רמזורים עירוניים והרחבה לערים נוספות.
- פריסה וניטור בענן – פריסת השרת ושירות ה-AI בקונטיינרים, ניטור תקינות והגבלת קצב בקשות.
- אפליקציה ניידת ייעודית לטכנאים עם ניווט והתראות Push.

קישורים

גיטהאב: [SafeCross](#)

סרטון הדגמה ב-Youtube: [קישור YouTube – יתווסף]

נספח א': [נספח א'](#)